Attention 入门答疑

两段「不太懂」的原文，逐句拆解

2026 年 4 月 23 日

目录

1 开场：你现在在哪一节 **3**

2 术语补丁：什么是「隐状态 s_k」？ **3**

2.1 一句话先记住 . . . 3

2.2 拿一个画面想 . . . 3

2.3 RNN 是怎么「写笔记」的 . . . 3

2.4 为什么图里要说「都留下来不要扔」 . . . 4

3 第一段：把宇宙压进 512 个数 **4**

3.1 原文 . . . 4

3.2 场景：旧式翻译模型怎么工作 . . . 5

3.3 第一段（背景问题）在说什么 . . . 5

3.4 第二段（朴素直觉）在说什么 . . . 5

3.5 这一节为什么重要 . . . 6

4 第二段：加权查表 **6**

4.1 原文 . . . 6

4.2 两个核心比喻：「记忆表」+「软查表」 . . . 7

4.3 第一段（背景问题）在说什么 . . . 7

4.4 第二段（朴素直觉）：四步软查表 . . . 7

4.5 为什么这一段叫「加权查表」 . . . 8

4.6 为什么强调「没有 argmax、没有硬选择」 . . . 8

4.7 「不需要人工对齐」是什么意思 . . . 8

5 补讲：什么是「条件语言模型」(CLM) **9**

5.1 先回忆：普通语言模型 (LM) 在干嘛 . . . 9

5.2 条件语言模型 (CLM) 多了什么 . . . 9

5.3 逐符号翻译 . . . 10

5.4 为什么写成「连乘」? . . . 10

5.5 LM 与 CLM 的对比 . . . 10

5.6 这一节怎么和 attention 衔接 11

6 补讲：多头注意力 (Multi-Head Attention) 和 QKV 的关系 11

6.1 先回忆一下「单头」QKV 在干嘛 11

6.2 多头的朴素直觉 12

6.3 多头的数学表达 12

6.4 你看的那张图里「reshape + transpose」到底在做什么 12

6.5 为什么「单头 vs 多头」参数量是一样的 13

6.6 多头 vs 单头：到底差在哪？ 13

6.7 多头和「以前的 QKV」是什么关系？ 14

7 把两段合起来一句话讲完 14

1 开场：你现在在哪一节

这两段都来自前一份讲义的 **第 2 章「Attention」**。顺序是：

1. §2.2 **固定编码器表示的瓶颈**
   ——先把痛点讲透：旧 seq2seq 把整句话压成一个固定向量，丢信息、还不能回头看。
2. §2.3 **Attention 的高层视角**
   ——再给解药：让解码器每一步自己挑该看源端的哪几个词。

第一段是「为什么需要 attention」，第二段是「attention 到底在干嘛」。下面分别拆开看。

补：在进入正文前，先把「隐状态 $\boldsymbol{s}_k$」这个最容易卡住的术语讲清楚——见后面的「术语补丁」一节。

2 术语补丁：什么是「隐状态 $\boldsymbol{s}_k$」？

后面两段都会反复出现 $\boldsymbol{s}_1, \boldsymbol{s}_2, \ldots, \boldsymbol{s}_m$。如果你不熟悉 RNN，可能会卡在「隐状态到底是个啥」上。这一节专门把这件事讲透，再回去看后面就顺了。

2.1 一句话先记住

一句话总结

隐状态 = RNN 在读到第 k 个词时，它「脑子里」当前的内部记忆向量。

记号 $\boldsymbol{s}_k$ 就是「读完第 1 到第 k 个词之后，RNN 内部的状态」，是一个 d 维向量（比如 $d = 512$）。

2.2 拿一个画面想

把 RNN 编码器想成一个**一边读、一边在便签纸上不停改写笔记**的人：

时刻 k	它刚读到的词	它当前的「笔记」（= 隐状态）
1	“I”	$\boldsymbol{s}_1$（只看了 “I” 之后的脑内状态）
2	“love”	$\boldsymbol{s}_2$（看了 “I love” 之后的脑内状态）
3	“you”	$\boldsymbol{s}_3$（看了 “I love you” 之后的脑内状态）

每一个 $\boldsymbol{s}_k$ 都是一个数值向量，编码的是「到目前为止读过的所有词留下的记忆」。源句有 m 个词，就有 m 个隐状态：$\boldsymbol{s}_1, \boldsymbol{s}_2, \ldots, \boldsymbol{s}_m$。

2.3 RNN 是怎么「写笔记」的

每读一个新词 $\boldsymbol{x}_k$，RNN 用一个固定的小函数 f 把「上一刻的笔记」和「这一刻新看到的词」合成新笔记：

RNN 的更新公式

$$\boldsymbol{s}_k = f(\boldsymbol{s}_{k-1}, \boldsymbol{x}_k).$$

- $\boldsymbol{s}_{k-1}$：上一时刻的笔记（旧记忆）；
- $\boldsymbol{x}_k$：第 k 个词的向量（新输入）；
- f：RNN/LSTM/GRU 内部那组带参数的小公式，全句子共用同一个 f；
- $\boldsymbol{s}_k$：更新后的新笔记。

注意：「隐藏」不是说「藏起来不让你看」，而是因为它不是网络的最终输出，只是中间的内部状态，所以叫 hidden state。

2.4 为什么图里要说「都留下来不要扔」

旧 seq2seq：它只把最后一张笔记 $\boldsymbol{s}_m$ 交给解码器，前面 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_{m-1}$ 全部扔掉。
$\Rightarrow$ 解码器只看到一句话的「最终摘要」，看不到细节。

Attention 的做法：所有 m 张笔记都留着，组成一个「记忆表」。解码器每生成一个目标词，就回过头在这 m 张笔记里挑该看哪几张、看多少。

直觉

所以「让编码器把每个时刻的隐状态 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ 都留下来，不要扔」翻译成大白话就是：
编码器一边读源句一边记笔记，每读一个词都生成一张当时的内部笔记 $\boldsymbol{s}_k$。旧做法只把最后一张笔记交出去；新做法（*attention*）要求把所有 m 张笔记都交出去，留给解码器后面慢慢按需查阅。

一句话总结

看到 $\boldsymbol{s}_k$ 就翻译成：「读到第 k 个源词时，编码器内部的那张数值化笔记」。
看到 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ 就翻译成：「整张源端笔记本」。

3 第一段：把宇宙压进 512 个数

3.1 原文

原文段落

2.2.1 背景问题：一个向量装不下整句话

经典 seq2seq 的编码器读完全句之后，吐出一个固定维度的向量 $\boldsymbol{h}_m^{\text{enc}} \in \mathbb{R}^d$，把它当作整句的「意思」交给解码器。但是：

- 句子是 5 个词还是 50 个词，最后给解码器的都是同一个 d 维向量。
- 解码器在生成第 1 个目标词时关心的源信息，和生成第 10 个目标词时关心的源信息，显然不一样，但它每一步看到的「源信息」都是同一个向量。

2.2.2 朴素直觉：把宇宙压进 512 个数

Lena 在博客里有一个非常形象的比喻：让编码器把整句话压成一个固定向量，就像让你把整个宇宙压进 512 个数再交出去。短句还行，长句就一定会丢信息。
更糟的是：解码器每一步都要从同一个「宇宙摘要」里翻出当前需要的那一小块信息——它根本没机会「回头再看一眼原文」。

3.2 场景：旧式翻译模型怎么工作

想象你要把一句英文翻成中文。旧式 seq2seq 是这样的：

1. **编码器**（一个 RNN）一个词一个词地读完整句英文。
2. 读完之后，它挤出一个固定大小的向量 $\boldsymbol{h}_m^{\text{enc}} \in \mathbb{R}^d$。这里 d 是预先设定好的（比如 512），**不管原句多长，这个向量的大小始终是 512**。
3. **解码器**（另一个 RNN）拿到这一个向量，开始一个字一个字地往外吐中文。

3.3 第一段（背景问题）在说什么

这个设计有两个尴尬：

尴尬一：句子长度差很多，但「摘要」只有同一个尺寸。

- 5 个词的话："I love you"
- 50 个词的话："Yesterday I went to the store and bought some bread and milk and …"

两种句子最后都被压成同一个 *512* 维向量交给解码器。
⇒ 长句子里的细节肯定塞不下，必然有信息丢失。

尴尬二：解码器每一步看到的「源信息」是同一个向量。

- 解码器吐第 1 个中文字时，它最关心英文里的开头几个词。
- 解码器吐第 10 个中文字时，它最关心英文里中间或末尾的词。

但旧设计里，解码器每一步都在看那个完全一样的 $\boldsymbol{h}_m^{enc}$。它**没有「现在我该多看一眼原文里第 7 个词」的能力**——因为原文已经不见了，只剩那个压扁的摘要。

3.4 第二段（朴素直觉）在说什么

Lena 用一个比喻让你直观感受到这有多荒谬：

直觉

让编码器把整句话压成一个固定向量，就像让你把整个宇宙压进 *512* 个数再交出去。

- 「宇宙」= 一句话里所有的词、语法、含义、上下文。
- 「512 个数」= 那个固定大小的向量。

短句子像「猫」「天空蓝」，500 多个数还够用；但「《红楼梦》第三回的剧情」要塞进 512 个数，必然丢一大堆。

更糟糕的是后半句：

> 解码器每一步都要从同一个「宇宙摘要」里翻出当前需要的那一小块信息——它根本没机会「回头再看一眼原文」。

意思是：哪怕你把宇宙压成了 512 个数，至少别拦着我回去翻原书啊！但旧模型就是不让你回去翻——你只能看摘要，原文已经被扔掉了。

3.5 这一节为什么重要

它是用来引出 *attention* 的痛点描述。下一节（也就是第二段那张图）就会讲：

> 既然「压成一个向量」这么难受，那干脆**别压了**——编码器把每个词的状态 $\boldsymbol{s}_1, \boldsymbol{s}_2, \ldots, \boldsymbol{s}_m$ 全部留着，解码器每一步自己挑该看哪几个，按权重加权求和。

这个「按需回头看原文」的机制，就是 **attention**。

一句话总结

先把「不能回头看原文」这件事的痛感讲透，下一节 attention 出场时你才会觉得「啊，确实需要它」。

4 第二段：加权查表

4.1 原文

原文段落

2.3.1 背景问题：能不能让解码器「回头看」？

既然「压成一个向量」是瓶颈，那很自然的想法是：

- 让编码器把每个时刻的隐状态 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ 都留下来，不要扔。
- 解码器在第 t 步，根据当前自己的状态 $\boldsymbol{h}_t$，自己决定该重点看源端的哪些位置。

关键约束是：这个「决定看哪里」的过程必须可微，否则没法用反向传播一起训练。

2.3.2 朴素直觉：加权查表

把编码器输出 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ 想成一张「源端记忆表」。解码器每一步做的事就是一次软查表：

1. 拿出一个「查询」$\boldsymbol{h}_t$（解码器当前状态）。
2. 对每条记忆 $\boldsymbol{s}_k$ 打一个相关性分数 e_{tk}。
3. 把分数过一个 softmax 得到「看每条记忆的比例」α_{tk}。
4. 按比例把所有记忆加权求和，得到这一步的「context」$\boldsymbol{c}_t$。

注意整个过程没有 argmax、没有硬选择，所以处处可导，可以端到端训练。模型自己会学出「在第 t 步该看哪里」，不需要我们提供人工对齐。

4.2 两个核心比喻：「记忆表」+「软查表」

直觉

把这一段的整个画面想成「图书馆查资料」：

- 编码器输出 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m \Rightarrow$ **书架**，每本书是源端的一个词；
- 解码器当前状态 $\boldsymbol{h}_t \Rightarrow$ 你心里那条**查询请求**：「我现在想找什么?」；
- 每个相关性分数 $e_{tk} \Rightarrow$ 第 k 本书**有多符合**你的请求；
- softmax 之后的 $\alpha_{tk} \Rightarrow$ 你打算**看每本书的多大比例**（加起来等于 1）；
- 加权求和得到 $\boldsymbol{c}_t \Rightarrow$ 你最终从这些书里读到的**综合答案**。

4.3 第一段（背景问题）在说什么

第 1 条「都留下来，不要扔」。 旧模型只把编码器的最后一个状态 $\boldsymbol{h}_m^{\text{enc}}$ 交给解码器，其它隐状态全丢了。新做法是把所有 m 个时刻的中间状态 $\boldsymbol{s}_1, \boldsymbol{s}_2, \ldots, \boldsymbol{s}_m$ 都保留。

- 旧：编码器 = m 个状态都算了，但只把最后 1 个交出去；
- 新：编码器 = m 个状态都算了，并且全部交出去。

第 2 条「自己决定该重点看哪些位置」。 解码器在生成第 t 个目标词时，它自己有一个内部状态 $\boldsymbol{h}_t$（「我当前想说的是这种意思」）。它用这个 $\boldsymbol{h}_t$ 当线索，去和每一条 $\boldsymbol{s}_k$ 比对，自动算出「这一步我要重点看 $\boldsymbol{s}_3$ 和 $\boldsymbol{s}_5$」。

关键约束：「必须可微」。 神经网络靠**反向传播**训练。反向传播要求每一步都可导：损失对参数的梯度要能一路传回去。**硬选择**（比如「我就只看第 3 条」）是离散的，导数等于 0 或不存在，梯度断了，模型学不会。所以必须用软选择：「第 3 条的权重 0.6，第 5 条的权重 0.3，其它共 0.1」。权重可以是任意实数 $\Rightarrow$ 处处可导 $\Rightarrow$ 反向传播能过去。

一句话总结

背景问题这一段的总结：
（1）原始信息全部留住；（2）解码器每步根据自己的状态自动加权看；（3）「加权」是为了可导，可导才能用神经网络的方法训练。

4.4 第二段（朴素直觉）：四步软查表

把抽象的话翻成数学，正好就是 4 行公式。设源端有 m 个词，编码器隐状态 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$；解码器在第 t 步的状态是 $\boldsymbol{h}_t$。

Attention 的四步

1. **准备查询：** 当前查询就是 $\boldsymbol{h}_t$。

2. **打相关性分数：** 对每个 $k = 1, \ldots, m$，

$$e_{tk} \;=\; \mathrm{score}(\boldsymbol{h}_t, \boldsymbol{s}_k).$$

score$(\cdot,\cdot)$ 可以是点积 $\boldsymbol{h}_t^\top \boldsymbol{s}_k$，也可以是 MLP，也可以是双线性 $\boldsymbol{h}_t^\top W \boldsymbol{s}_k$。

3. **归一化成「看的比例」：**

$$\alpha_{tk} \;=\; \frac{\exp(e_{tk})}{\sum_{j=1}^{m} \exp(e_{tj})} \;=\; \mathrm{softmax}_k(e_{tk}).$$

这一步保证 $\sum_k \alpha_{tk} = 1$ 且每个都非负，解释成「这一步看第 k 个源词的比例」。

4. **加权求和得到 context：**

$$\boldsymbol{c}_t \;=\; \sum_{k=1}^{m} \alpha_{tk}\, \boldsymbol{s}_k.$$

$\boldsymbol{c}_t$ 就是这一步专门为第 t 个目标词**量身定制**的源端摘要。

4.5 为什么这一段叫「加权查表」

把它和「在 Python 字典里查一个 key」对比一下，最容易看清楚：

普通查表	Attention（软查表）	关键差别
key 必须完全相等	用打分函数 e_{tk} 算相似度	软的
只返回 1 个 value	返回所有 value 的加权和	软的
判断「相等」是离散的	softmax 是连续可导的	可训练的

「软查表」三个字精准地概括了这件事：**比对** + **加权** + **求和**，全程可导。

4.6 为什么强调「没有 argmax、没有硬选择」

直觉

如果模型在第 t 步硬性「挑一个」最相关的源词，那就是 argmax，对应「我只看第 3 条记忆」。argmax 是阶跃函数，导数几乎处处为 0。

后果： 模型不知道「应该把分数稍微挪一点点会变好还是变坏」，反向传播完全没法告诉它该怎么改参数。

软查表的妙处： 权重是连续的实数。权重稍微变化，最后的 $\boldsymbol{c}_t$ 也会跟着平滑变化，损失也跟着平滑变化，所以梯度有方向感，模型才能慢慢学出「在第 t 步看哪里更好」。

4.7 「不需要人工对齐」是什么意思

「**对齐（alignment）**」是机器翻译里的传统术语，指「源句的第 k 个词对应到目标句的第 t 个词」。统计翻译时代要专门用算法（如 IBM Model 1/2/3）去估对齐，甚至请人工标注。

而 attention 不需要——我们只给模型「源句」和「目标句」这两端的对照，模型在训练损失（生成目标句的对数似然）的驱动下，会自动让那些 α_{tk} 在「正确对应位置」上变大。这就是「学出来了对齐」，而不是被告知对齐。

一句话总结

朴素直觉这一段就是把上面背景问题里两条诉求（「都留下来」+「自己决定看哪里」+「必须可微」）精确写成 4 步：**查询** → **打分** → **softmax** → **加权求和**。

5 补讲：什么是「条件语言模型」(CLM)

这一段你看的是 §1.3。它在解释一个概念：**seq2seq 的 decoder 其实就是一个特殊的语言模型**。但很多人卡在「条件」这两个字上，所以在这里专门展开。

5.1 先回忆：普通语言模型 (LM) 在干嘛

直觉

普通语言模型只做一件事：

> 给我前面写过的几个词，告诉我下一个词最可能是什么。

它没有任何外部输入——没人给它一张图、一句英文，它就是在「凭空续写」。

数学上，对一句话 $\boldsymbol{y} = (y_1, y_2, \ldots, y_T)$，LM 把整句的概率拆成「左到右一个一个写」的链式乘积：

普通语言模型的概率分解

$$p(\boldsymbol{y}) = \prod_{t=1}^{T} p(y_t \mid y_{<t}), \qquad y_{<t} := (y_1, \ldots, y_{t-1}).$$

读法：「整句出现的概率 = 一个个词依次出现的条件概率连乘起来。」其中 $y_{<t}$ 只是一个简写，意思是「截至第 t 步之前已经写出来的所有词」。

5.2 条件语言模型 (CLM) 多了什么

直觉

条件语言模型只比普通 LM 多一样东西：**一个外部条件 $\boldsymbol{x}$**。

- 翻译里，$\boldsymbol{x}$ 是源句（英文）；
- 摘要里，$\boldsymbol{x}$ 是原文；
- 图像描述里，$\boldsymbol{x}$ 是一张图。

它的工作变成：

> 给我前面写过的几个词，**以及一份外部材料 $\boldsymbol{x}$**，告诉我下一个词最可能是什么。

写成公式只是把每一个条件概率都「再加一个 $\boldsymbol{x}$」：

条件语言模型的概率分解

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \prod_{t=1}^{T} p(y_t \mid y_{<t}, \boldsymbol{x}).$$

5.3 逐符号翻译

很多人卡在公式的符号上，把每个符号翻成中文：

符号	翻译
$\boldsymbol{x}$	源端给的「材料」(源句、图片、文档……)
$\boldsymbol{y}$	我要写出来的目标句 $(y_1, \ldots, y_T)$
y_t	目标句第 t 个词
$y_{<t}$	截至第 t 步之前已写出的所有词
$p(y_t \mid y_{<t}, \boldsymbol{x})$	在「已经写了什么」+「材料是啥」给定的条件下，第 t 个词的概率
$\prod_{t=1}^{T}$	把 T 个词的条件概率全部相乘
$p(\boldsymbol{y} \mid \boldsymbol{x})$	在材料 $\boldsymbol{x}$ 给定的情况下，写出整句 $\boldsymbol{y}$ 的概率

5.4 为什么写成「连乘」？

直觉

因为 decoder 是**一个词一个词写**出来的：先写 y_1，再用 y_1 当历史去写 y_2，再用 (y_1, y_2) 当历史去写 y_3……
每写一步都是一次「在已有历史下挑下一个词」，所以整句的概率就是这些「每一步条件概率」的乘积。

链式法则告诉我们：任何联合概率都可以分成这样的连乘。这里只是把这条普适的概率法则套到「按时间顺序生成」这个特殊场景上。

5.5 LM 与 CLM 的对比

	普通 LM	条件 LM (= seq2seq decoder)
建模目标	$p(\boldsymbol{y})$	$p(\boldsymbol{y} \mid \boldsymbol{x})$
有没有外部输入	没有	有，记作 $\boldsymbol{x}$
每一步在算啥	$p(y_t \mid y_{<t})$	$p(y_t \mid y_{<t}, \boldsymbol{x})$
经典任务	续写、文本生成	翻译、摘要、问答、图像描述

一句话总结

seq2seq = 条件语言模型。
和普通 LM 在结构上几乎一样，唯一差别是：「写每个词的时候，它可以额外看一眼源端材料 $\boldsymbol{x}$」。

5.6 这一节怎么和 attention 衔接

普通 LM 看「自己已经写了什么」就够了。CLM 多了 $\boldsymbol{x}$ 之后，问题就变成：

怎么把 $\boldsymbol{x}$ 传给 decoder？

最早的答案：把 $\boldsymbol{x}$ 用 RNN 编码成一个固定向量 $\boldsymbol{h}_m^{\text{enc}}$，丢给 decoder 当起点——这就是 §2.2 那个「装不下整句话」的方案。

后来的答案：把 $\boldsymbol{x}$ 编码成一整张笔记 $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$，让 decoder 每写一个词都用 attention 临时挑一份——这就是 §2.3 的「软查表」方案。

所以这条链是：

$$\textbf{LM} \;\rightarrow\; \textbf{CLM} \;\rightarrow\; \textbf{怎么传 } \boldsymbol{x} \;\rightarrow\; \textbf{Attention}$$

6 补讲：多头注意力 (Multi-Head Attention) 和 QKV 的关系

这一段是 Transformer 的多头注意力。你的疑问其实就是：「**QKV 我大概懂了，那多头到底多在哪？参数还是那一份吗？还是真的开了 h 套？**」下面把这件事彻底讲透。

6.1 先回忆一下「单头」QKV 在干嘛

回顾 §2.3 那张「软查表」：

1. 把每个 token 的向量分别投影成三份角色：$Q = XW_Q$，$K = XW_K$，$V = XW_V$；
2. 用 Q 去和 K 打分、做 softmax，得到「我看每个 token 的比例」；
3. 按比例加权求和 V，得到输出。

单头缩放点积注意力

$$\text{Attention}(Q, K, V) \;=\; \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

形状（$X \in \mathbb{R}^{n \times d_{\text{model}}}$，例如 $d_{\text{model}} = 512$）：
$Q, K, V \in \mathbb{R}^{n \times d_{\text{model}}}$，输出也是 $n \times d_{\text{model}}$。

单头的局限：一次注意力只能学出一种「看哪里」的规律。但语言里同时存在多种关系：

- 主语 ↔ 谓语；
- 形容词 ↔ 它修饰的名词；
- 代词 ↔ 它指代的对象；
- 短语边界 / 标点 / 邻居……

一个头很难同时管这么多种「应该看哪」。

6.2 多头的朴素直觉

直觉

多头 = 同时开 h 套「看法」，每套独立地学一种关注模式，最后把它们的结果拼起来。
你可以想成：与其请一个全能眼睛同时盯所有关系，不如请h 个各看一种的小眼睛同时看，最后把它们写的笔记拼成一份大笔记。

6.3 多头的数学表达

设有 h 个头（典型 $h = 8$），每个头自己有三组小投影矩阵 W_Q^i, W_K^i, W_V^i $(i = 1, \ldots, h)$，还有一个公用的输出投影 W_O：

Multi-Head Attention

$$\text{head}_i \;=\; \text{Attention}(QW_Q^i,\; KW_K^i,\; VW_V^i),$$

$$\text{MultiHead}(Q, K, V) \;=\; \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O.$$

其中 $W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d_{\text{model}} \times d_k}$，$W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$，$d_k = d_v = d_{\text{model}}/h$。

把它和单头对比起来看，差别就两件事：

1. 投影矩阵从「*1* 套大」W_Q, W_K, W_V 变成「h 套小」W_Q^i, W_K^i, W_V^i；每套把 d_{model} 投到**更小**的 $d_k = d_{\text{model}}/h$ 维。
2. 算完 h 份 head 输出，拼起来再过一个输出投影 W_O。

备注 1. 尺寸的关键约束： $d_k = d_{model}/h$。
这意味着「每个头看的维度变小了」，但「头数变多了」，两者乘起来刚好回到 d_{model}——所以总尺寸守恒。$h = 8, d_{model} = 512$ 时，每个头只有 64 维。

6.4 你看的那张图里「reshape + transpose」到底在做什么

图里那段「代码 / 工程对应」最容易让人懵。其实它在解决一个工程问题：别真的开 h 个 *for* 循环跑 h 次注意力，那样太慢。聪明的做法是**一次大矩阵乘 + 一次 reshape**，让 GPU 一口气把 h 个头并行算掉。

形状变换的逐步图解

设 batch 大小 B，序列长度 n，$d = d_{\text{model}}$，$d_k = d/h$。

1. **一个大投影做完所有头的 Q：** $X \in \mathbb{R}^{B \times n \times d} \xrightarrow{W_Q \in \mathbb{R}^{d \times d}} Q \in \mathbb{R}^{B \times n \times d}$。
   看起来还是 d 维，但其实里面已经塞着 h 段独立的 d_k 维向量了。
2. **reshape 把最后一维拆成 (h, d_k)：** $\mathbb{R}^{B \times n \times d} \to \mathbb{R}^{B \times n \times h \times d_k}$。
3. **transpose 让 h 跑到第二位：** $\mathbb{R}^{B \times n \times h \times d_k} \to \mathbb{R}^{B \times h \times n \times d_k}$。
   现在 h 维充当一个新的「批量轴」——每个头各算各的，互不干扰。

4. **对 K、V 做完全一样的三步**，得到三个 $B \times h \times n \times d_k$ 的张量。
5. **一次 batched matmul 同时算完 h 个头的 attention：**

$$\text{scores} = QK^\top \in \mathbb{R}^{B\times h\times n\times n}, \quad \text{out} = \text{softmax}(\text{scores}/\sqrt{d_k})\,V \in \mathbb{R}^{B\times h\times n\times d_k}.$$

6. **transpose 回去 + reshape 拼接 + 过 W_O：** $\mathbb{R}^{B\times h\times n\times d_k} \to \mathbb{R}^{B\times n\times h\times d_k} \to \mathbb{R}^{B\times n\times d} \xrightarrow{W_O} \mathbb{R}^{B\times n\times d}$。

「*reshape 让 h 变成 batch 轴*」的意思就是第 3 步：原来 h 是「同一份数据里的不同切片」，通过 transpose 变成「批量维度里的并行样本」，GPU 就可以一次性算 h 份注意力。

6.5 为什么「单头 vs 多头」参数量是一样的

很多人第一次看会觉得「多头是不是参数变 h 倍?」答案：**不是**。来对比一下：

	单头（用 d_{model} 做注意力）	多头（h 个头，每头 $d_k = d_{\text{model}}/h$）
W_Q 的形状	$d_{\text{model}} \times d_{\text{model}}$	h 个 $d_{\text{model}} \times (d_{\text{model}}/h)$
W_Q 的总参数	d_{model}^2	$h \cdot d_{\text{model}} \cdot (d_{\text{model}}/h) = d_{\text{model}}^2$
W_K, W_V	同上 d_{model}^2 各一	同上 d_{model}^2 各一
W_O	没有（或视作恒等）	d_{model}^2

主体的 W_Q, W_K, W_V 在两种写法下总参数完全一样，都是 $3d_{\text{model}}^2$。多头只是把同一份参数切成 h 份，让每一片只看一个 d_k 维子空间。

直觉

所以正确的画面是：**多头不是「叠 h 个独立的网络」，而是「把同一份参数切成 h 块去看不同维度」**。增加的只有最后一个 W_O（输出投影），用来重新融合 h 段。

6.6 多头 vs 单头：到底差在哪？

	单头	多头
注意力矩阵数量	1 张 $(n \times n)$	h 张 $(n \times n)$
每个头看几维?	d_{model}	$d_k = d_{\text{model}}/h$
能学几种关注模式?	1 种	h 种（实际上每个头会专攻一类关系）
总参数量	$\approx 3d_{\text{model}}^2$	$\approx 4d_{\text{model}}^2$（多了 W_O）
GPU 并行	已并行	仍然并行（reshape 后 h 当 batch 轴）

6.7 多头和「以前的 QKV」是什么关系？

直接回答你的疑问：

一句话总结

多头 = 把以前那一份 QKV 在「特征维」上切成 h 份，每一份独立做注意力，最后拼回来再过一个 W_O。
QKV 这套机制完全没有变，只是**在哪一段子空间上做**从「整段 d_{model}」变成「h 段各 d_k」，每段学一种「看哪里」的规律。

形象一点：

- 单头 QKV 像一**只眼睛**盯着 512 维全空间；
- 多头 QKV 像 **8 只眼睛**各盯一个 64 维小空间，看完写各自的笔记，最后用 W_O 把 8 段笔记统一润色成一份完整的 512 维输出。

最后注意一点：图里说「单头和多头参数量一样」，严格讲是主体投影 W_Q, W_K, W_V 一样；多头多出来的是 W_O。但和模型其它部分（FFN 等）相比，这点开销几乎不计。

7 把两段合起来一句话讲完

1. §2.2 告诉你：旧 seq2seq 把整句压成一个固定向量，**装不下、还不能回头看**——这是病。
2. §2.3 告诉你：药方就是**软查表**——编码器把所有时刻的隐状态都留着当「记忆」，解码器每一步用自己的状态当「查询」，给每条记忆打一个 softmax 权重，最后加权求和得到这一步专属的源端 context $\boldsymbol{c}_t$。
3. 这件事处处可导 ⇒ 可以**端到端**训练，模型自己学出「该看哪里」，不需要人工对齐。

一句话总结

Attention 的本质就一句话：
不再压缩源端信息，而是在每一步临时按需重新组合它。